

Ronin Kernel v3.0: Hardened Architecture Blueprint

Target System: Android Native Edge AI Runtime **Reference SoC:** Snapdragon 778G / 778G+ (Mi 11 Lite 5G NE Level) **Status:** Hardened Specification (Phase 1 Baseline) **Base Philosophy:** Derive More, Store Less & Zero-Memory-Fragmentation

၁။ System Architecture Overview (The Memory Fabric Matrix)

Ronin Kernel v3.0 သည် Point-to-Point Coupling စနစ်များကို လုံးဝဖျက်သိမ်းပြီး **"Everything is a Memory Event"** ဟူသော မူဝါဒဖြင့် In-Process Memory Fabric Matrix အဖြစ် ပြောင်းလဲစဉ်းသည်။ ကိရိယာအတွင်း ဖြစ်ပျက်သမျှ Input/Output တိုင်းကို ပေါ့ပါးသော C++ Struct Contract များအဖြစ် ပြောင်းလဲ၍ Lock-Free Internal Pipeline ထဲသို့ စီးဆင်းစေသည်။

[Chat Event] -----> [Lock-Free Queue] -----> [Memory Fabric Router]

[Sensor DSP] -----> (Pointer-Based) |

[Tool Result] -----> (Fixed 8 bytes/entry) |

-----|-----

| [Gemma 4 KV Cache Injector]

[Buffered SQLite Ring] (Immediate / RAM)

(FTS5 Core / Storage)

| [

Inference Guard-rail]

(PAUSED during token gen)

၂။ Core Component Specifications

2.1. Polymorphic Payloads & Fixed-Size Event Struct

Memory Allocation Noise ကို လျော့ချရန်နှင့် Cache Alignment ကို ကောင်းမွန်စေရန်အတွက် Queue Entry တိုင်းကို Fixed 8-byte Entry Size (64-bit Pointer) ဖြင့်သာ တည်ဆောက်သည်။

- IMemoryPayload (Interface):** Virtual destructor နှင့် getType() / getMemorySize() functions များသာ ပါဝင်သော Pure Abstract Interface။
- MemoryEvent (Struct):** Meta-information (Timestamp, Importance Score, Event Type) နှင့် Payload Interface သို့ ညွှန်ပြထားသော std::shared_ptr<IMemoryPayload> သာ

ပါဝင်ရမည်။

2.2. Deterministic Memory Payload Object Pool (Anti-Fragmentation)

Sensor Burst (50Hz+) သို့မဟုတ် Chat Streaming အချိန်များတွင် Memory Heap ပေါ်၌ new သို့မဟုတ် std::make_shared ခေါ်ယူခြင်းကို လုံးဝ တားမြစ်သည်။ JNI Initialization အဆင့်တွင် သတ်မှတ်ထားသော RAM Budget အတွင်း၌ Fixed-size Object Pools များကို တစ်ပြိုင်နက် Pre-allocate လုပ်ထားရမည်။

Payload Type	Struct Size (Fixed Block)	Max Slots	Allocated Memory	Memory Management Rule
SensorAnalysis	64 bytes	128 slots	8 KB	Fixed Pool Only (Drop on Full)
ChatMessage	4 KB	16 slots	64 KB	Option C: Fixed Pool + Heap Fallback
ToolResult	512 bytes	64 slots	32 KB	Fixed Pool Only
Total RAM Budget	—	—	104 KB	Target: < 200 KB (Safe for LMK)

- **Custom Lock-Free Deleter:** std::shared_ptr ၏ Refcount သည် ဖြစ်သွားသည့်အခါ OS Heap သို့ ပြန်မကျဘဲ Lock-Free Ring Buffer မှတစ်ဆင့် Object Pool ဆီသို့ $O(1)$ စွမ်းဆောင်ရည်ဖြင့် Auto-return ဖြစ်စေရမည်။ Gemma Inference Thread ကို Block ဖြစ်စေမယ့် Mutex Lock များ လုံးဝ မသုံးရ။
- **Option C Overflow Fallback:** မြန်မာစာ UTF-8 (1 Character = 3 bytes) Long Context များကြောင့် ChatMessage သည် 4KB ထက် ကျော်လွန်သွားပါက စနစ် Crash မဖြစ်စေရန်အတွက် ယာယီ Heap Allocation (std::make_shared) ကို ခွင့်ပြုပြီး System Health Log ထဲသို့ အသိပေးချက် ပို့ရမည်။

2.3. Lock-Free Bounded Queue & Backpressure Drop Policy

- **Queue Engine:** moodycamel::ConcurrentQueue<MemoryEvent> ကို အသုံးပြုပြီး အမြင့်ဆုံး Cap ကို 256 Events အဖြစ် ကန့်သတ်သည်။
- **Priority-Aware Drop Strategy ($O(1)$):** Queue ပြည့်သွားသည့်အခါ entries တစ်ခုချင်းစီကို လိုက်လံ Scan ဖတ်ခြင်း ($O(n)$ Anti-pattern) ကို ရှောင်ရှားရန် အောက်ပါ ဗျူဟာကို သုံးသည်:
 1. အသစ်ဝင်လာသော Event ၏ importance_score သည် Threshold (< 0.3) အောက် ရောက်နေပါက တိုက်ရိုက် **Instant Drop** လုပ်သည်။
 2. Threshold ထက် ကျော်လွန်ပါက Oldest Cold/Forgotten Event များကို **FIFO Band** အတိုင်း မဆိုင်းမတွ မောင်းထုတ် (Evict) ပစ်သည်။

2.4. Compile-Time Retention Policy Static Map

- Memory Lifecycles (ACTIVE, COLD, ARCHIVED, FORGOTTEN) စီမံခန့်ခွဲမှုအတွက် yaml-cpp သို့မဟုတ် JSON ကဲ့သို့သော Runtime String Parsing Library များကို လုံးဝ အသုံးပြုခြင်း မပြုရ။
- C++ constexpr Static Map နှင့် Template Traits Logic (Switch-case Optimization) ကို သုံးပြီး CPU Register Level တွင် < 1μs Lookup Time ဖြင့် Memory Demotion Rules များကို ဆုံးဖြတ်ရမည်။

2.5. Inference-Aware SQLite Writer Boundary

- **The Issue:** Snapdragon 778G ပေါ်တွင် Local Gemma 4 သည် NPU/GPU Acceleration ကို အပြည့်သုံး၍ Token ထုတ်နေချိန် (Inference Processing) တွင် Background မှ SQLite FTS5 Transaction များကို 100ms တစ်ခါ ဝင်ရောက် Commit လုပ်ပါက UFS Storage Controller ပေါ်တွင် I/O Jitter ဖြစ်ပြီး Model Output တွင် စကားလုံးများ တုံ့ခနဲ Stuttering ဖြစ်စေသည်။
- **The Hardening:** InferenceStateListener ဖြင့် Gemma ၏ State ကို စောင့်ကြည့်မည်။
 - is_inference_active() == true ဖြစ်နေချိန်တွင် SQLite Write Queue ထဲသို့ Data များကို RAM ပေါ်၌သာ ခေတ္တထိန်းထားပြီး **Disk Flush လုပ်ခြင်းကို လုံးဝ Pause/Defer ထားရမည်။**
 - is_inference_active() == false (Model Idle State) ရောက်မှသာ စုဆောင်းထားသော Event များကို Bulk Transaction အဖြစ် SQLite ထဲသို့ တစ်ချက်တည်း Flush လုပ်ရမည်။

၃။ C++ Structural Implementation Skeletons

3.1. memory_payload.h

```
#ifndef RONIN_MEMORY_PAYLOAD_H
#define RONIN_MEMORY_PAYLOAD_H

#include <string>
#include <cstdint>

enum class PayloadType : uint8_t {
    CHAT_MESSAGE,
    SENSOR_ANALYSIS,
    TOOL_RESULT
};

class IMemoryPayload {
public:
    virtual ~IMemoryPayload() = default;
    virtual PayloadType getType() const noexcept = 0;
    virtual size_t getMemorySize() const noexcept = 0;
};

// Concrete Types
struct ChatMessage : public IMemoryPayload {
    std::string role;
    std::string content;

    PayloadType getType() const noexcept override { return
PayloadType::CHAT_MESSAGE; }
    size_t getMemorySize() const noexcept override {
        return sizeof(ChatMessage) + content.capacity() +
role.capacity();
    }
};
```

```

    }
};

struct SensorAnalysis : public IMemoryPayload {
    float resonance_hz;
    float psd_db;
    bool is_anomaly;

    PayloadType getType() const noexcept override { return
PayloadType::SENSOR_ANALYSIS; }
    size_t getMemorySize() const noexcept override { return
sizeof(SensorAnalysis); }
};

struct ToolResult : public IMemoryPayload {
    std::string tool_name;
    std::string result_json;

    PayloadType getType() const noexcept override { return
PayloadType::TOOL_RESULT; }
    size_t getMemorySize() const noexcept override {
        return sizeof(ToolResult) + tool_name.capacity() +
result_json.capacity();
    }
};

enum class MemoryState : uint8_t { ACTIVE, COLD, ARCHIVED, FORGOTTEN
};

struct MemoryEvent {
    PayloadType type;
    int64_t timestamp_ms;
    float importance_score;
    std::shared_ptr<IMemoryPayload> payload;
};

#endif // RONIN_MEMORY_PAYLOAD_H

```

3.2. lockfree_payload_pool.h

```

#ifndef RONIN_LOCKFREE_PAYLOAD_POOL_H
#define RONIN_LOCKFREE_PAYLOAD_POOL_H

#include "memory_payload.h"
#include <concurrentqueue.h> // moodycamel lock-free queue
#include <memory>
#include <vector>

```

```

#include <atomic>

class MemoryPayloadPool;

struct PoolDeleter {
    MemoryPayloadPool* pool_ptr;
    void operator()(IMemoryPayload* ptr) const noexcept;
};

class MemoryPayloadPool {
private:
    moodycamel::ConcurrentQueue<ChatMessage*> chat_slots;
    std::vector<ChatMessage> chat_buffer; // Contiguous block

    moodycamel::ConcurrentQueue<SensorAnalysis*> sensor_slots;
    std::vector<SensorAnalysis> sensor_buffer;

    std::atomic<uint64_t> pool_overflow_count{0};

public:
    MemoryPayloadPool(size_t chat_count = 16, size_t sensor_count =
128) {
        // Pre-allocate Chat Slots (Option B Constraint)
        chat_buffer.resize(chat_count);
        for (size_t i = 0; i < chat_count; ++i) {
            chat_buffer[i].content.reserve(4096); // 4KB Reserved for
Myanmar UTF-8
            chat_slots.enqueue(&chat_buffer[i]);
        }

        // Pre-allocate Sensor Slots
        sensor_buffer.resize(sensor_count);
        for (size_t i = 0; i < sensor_count; ++i) {
            sensor_slots.enqueue(&sensor_buffer[i]);
        }
    }

    std::shared_ptr<ChatMessage> acquire_chat() {
        ChatMessage* ptr = nullptr;
        if (chat_slots.try_dequeue(ptr)) {
            ptr->content.clear();
            ptr->role.clear();
            return std::shared_ptr<ChatMessage>(ptr,
PoolDeleter{this});
        }
        // Option C Rule: Heap Fallback for Overflow
        pool_overflow_count.fetch_add(1, std::memory_order_relaxed);
        auto fallback =

```

```

std::get_deleter<void>(std::shared_ptr<ChatMessage>());
    return std::make_shared<ChatMessage>();
}

void return_chat_lockfree(ChatMessage* ptr) noexcept {
    // Check boundary to see if it belongs to pre-allocated buffer
    if (ptr >= &chat_buffer.front() && ptr <= &chat_buffer.back())
    {
        chat_slots.enqueue(ptr);
    } else {
        delete ptr; // Standard OS cleanup for dynamic fallback
object
    }
}
};

inline void PoolDeleter::operator()(IMemoryPayload* ptr) const
noexcept {
    if (!ptr) return;
    if (ptr->getType() == PayloadType::CHAT_MESSAGE) {

pool_ptr->return_chat_lockfree(static_cast<ChatMessage*>(ptr));
    }
    // Handle other types accordingly...
}

#endif // RONIN_LOCKFREE_PAYLOAD_POOL_H

```

3.3. constexpr_retention.h

```

#ifndef RONIN_CONSTEXPR_RETENTION_H
#define RONIN_CONSTEXPR_RETENTION_H

#include "memory_payload.h"

struct RetentionConfig {
    MemoryState initial_state;
    uint32_t demote_to_cold_after_sec;
    uint32_t archive_after_sec;
    bool archive_if_anomaly;
};

class RetentionPolicy {
public:
    // Zero Parsing Cost, Compile-Time Optimized Lookups
    static constexpr RetentionConfig getRule(PayloadType type)
noexcept {

```

```

switch (type) {
    case PayloadType::CHAT_MESSAGE:
        return RetentionConfig{MemoryState::ACTIVE, 300, 3600,
false};
    case PayloadType::SENSOR_ANALYSIS:
        return RetentionConfig{MemoryState::COLD, 0, 86400,
true};
    case PayloadType::TOOL_RESULT:
        return RetentionConfig{MemoryState::FORGOTTEN, 0, 600,
false};
}
};

#endif // RONIN_CONSTEXPR_RETENTION_H

```

၄။ Phase 1 Testing & Verification Protocols

ဗိသုကာ၏ ခိုင်မာမှုကို စစ်ဆေးရန် GoogleTest (GTest) ဖြင့် အောက်ပါ ယူနစ်တက်စ် (၃) ခုကို မဖြစ်မနေ အောင်မြင်ရမည်။

1. **Queue Memory Footprint Verification:** MemoryEvent ၏ Size အား စစ်ဆေးရန်။ Pointer-based ဖြစ်ရမည်ဖြစ်ပြီး sizeof(MemoryEvent) သည် Payload အမျိုးအစားပြောင်းသော်လည်း ပြောင်းလဲမှုမရှိဘဲ တည်ငြိမ်နေရမည်။
2. **Object Pool Stress Test:** Simulated Sensor Events ၁၀,၀၀၀ ကို 50Hz frequency ဖြင့် စဉ်ဆက်မပြတ် Queue ထဲသို့ ပစ်ထည့်မည်။ ထို Hot Path Hot-run အတွင်း malloc/free Allocation Count သည် == 0 ဖြစ်ရမည်။ (Heap Fragmentation ကာကွယ်ရေး အောင်မြင်မှုအား စစ်ဆေးခြင်း)။
3. **Inference-Aware Storage Block Check:** Atomic Flag is_inference_active ကို true ဟု သတ်မှတ်ထားချိန်တွင် SQLite Database Core ဆီသို့ Write Operation တောင်းဆိုမှုများကို Defer လုပ်/မလုပ်နှင့် Thread Blocking ရှိ/မရှိ Stress Test ပြုလုပ်ရန်။